CoreML: Implementing the Transformer and the Variations.

Bhuvanesh Reddy

May 20, 2026

Abstract. I present a from-scratch implementation of the Baseline Transformer architecture along with exploring variants in the positional encoding alongside Absolute Positional Encoding such as Relative Positional Encoding, Rotary Positional Encoding and ALiBi. I also explore variations in the attention mechanism such as Multi-Query Attention, Grouped-Query Attention and Softmax-Free attention. Moreover, I have also done Attention + Convolution hybrids, covering the different ways of integrating convolution into the attention architecture. Finally I have logged the training dynamics of all the models and compared them against each other and analyzed the results and picked the best performing combination.

1. Introduction.

The Transformer architecture has revolutionalized natural language processing. Unlike traditional RNN-based models, the Transformer uses *self-attention* mechanisms to capture relationships between tokens in a sequence. It has been done in a way to maximize parallelization, allowing for much efficient training on large datasets. The original Transformer architecture consists of an encoder-decoder structure, where both the encoder and decoder are composed of multiple layers of multi-head self-attention and feed-forward networks. The encoder processes the input sequence, while the decoder generates the output sequence one token at a time, attending to both the encoder's output and its own previous outputs. But I have implemented a decoder-only architecture in this assignment, which is commonly used for language modelling tasks.

2. Architecture. (Baseline)

2.1 Self-Attention Mechanism.

Given, Queries $\mathbf{Q} \in \mathbb{R}^{L\times d_k}$, Keys $\mathbf{K} \in \mathbb{R}^{L\times d_k}$, and Values $\mathbf{V} \in \mathbb{R}^{L\times d_k}$, which are linear projections of input $\mathbf{X} \in \mathbb{R}^{L\times d_{\text{model}}}$, the attention weights are computed as:

$$\text{Self Attention}\left(\mathbf{Q},\mathbf{K},\mathbf{V}\right) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

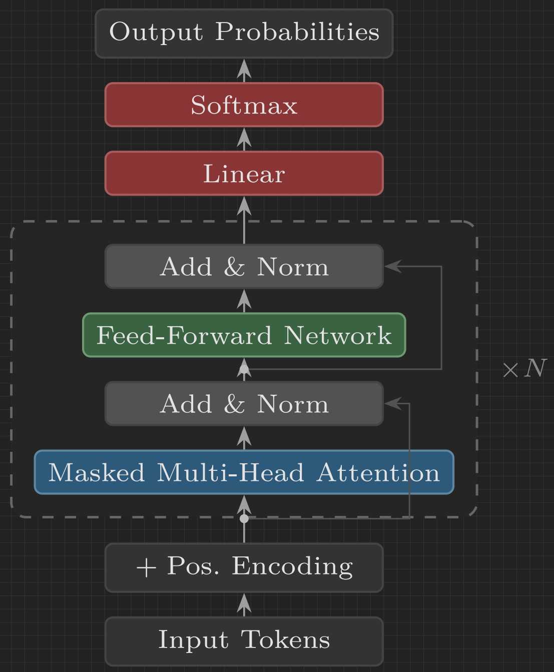

Figure 1. Decoder-only Transformer. Dashed block stacked N times.

Definition — Multi-Head Attention.

In Multi-Head Attention, we have h parallel attention heads, each with its own set of learned linear projections for queries, keys, and values. The dimensions of the projections are such that,

$$d_k = \frac{d_{\text{model}}}{h}$$

Each head computes attention independently, and their outputs are concatenated and go through another linear projection, $\mathbf{W}^O \in \mathbb{R}^{d_{\text{model}}\times d_{\text{model}}}$. Formally, the multi-head attention can be expressed as:

$$\text{MHA}\left(\mathbf{Q},\mathbf{K},\mathbf{V}\right) = \text{Concat}\left(\text{head}_1,\ldots,\text{head}_h\right)\mathbf{W}^O$$

where $\text{head}_i = \text{Attention}\left(\mathbf{Q}_i,\mathbf{K}_i,\mathbf{V}_i\right)$.

Remark: Masking.

In the attention weight matrix, we apply a mask to prevent positions from attenting to subsequent tokens, ensuring that the prediction for position i can only depend on the known outputs at positions less than i. This is typically implemented by adding a large negative value (e.g., $-\infty$) to the masked positions before applying the softmax function, effectively setting their attention weights to zero.

$$\text{Mask} = \begin{cases} 0 & \text{if } j \leq i \\ -\infty & \text{if } j > i \end{cases}$$

2.2 Positional Encoding.

Since the self-attention mechanism is position invariant, we need to inject some information about the positioning of tokens in the sequences. The original Transformer paper uses *absolute positional encoding* (AbsPE) which adds a fixed sinusoidal encoding to the input embeddings. For a position *pos* and dimension i, the positional encoding is defined as:

$$\mathbf{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

$$\mathbf{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

Important — Implication of the Notation.

The $2i$ and $2i+1$ indices indicate that the positional encoding alternates between sine and cosine functions across the dimensions of the embedding i.e., for even indices, it uses the sine function, and for odd indices, it uses the cosine function. This design allows the model to capture both absolute and relative positional information, as the combination of sine and cosine functions can represent a wide range of positional relationships.

3. Architecture (P.E Variations).

Why is there a need of alternative positional encoding schemes? AbsPE has some limitations such as it does not generalize well to longer sequences than those seen during training, and it does not capture relative positional information effectively. Moreover, they waste a lot of parameters on encoding absolute position patterns. For example, usually an adjective is followed by a noun, which is a relative positional pattern i.e., it doesn't matter whether the adjective is at position 5 and noun is at position 6 or whether the adjective is at position 50 and noun is at position 51, the pattern still holds. AbsPE would need to learn separate encodings for each of these positions, which can be inefficient and may not generalize well to unseen sequence lengths. This motivates the need for alternative positional encoding schemes that can capture relative positional information more effectively and generalize better to longer sequences. Mathematically,

$$\mathbf{X} = X_{\text{token}} + X_{\text{PE}}$$

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V$$

In each, (lets take $\mathbf{Q}$ as example)

$$\mathbf{Q} = (X_{\text{token}} + X_{\text{PE}})\,\mathbf{W}^Q =$$

$$\mathbf{Q} = X_{\text{token}}\mathbf{W}^Q + X_{\text{PE}}\mathbf{W}^Q$$

The same happens with $\mathbf{K}$ and $\mathbf{V}$. This goes to show that AbsPE forces the model to learn separate representations for token content and positional information, which can lead to inefficiencies and poor generalization.

3.1 Rotary Positional Encoding. (RoPE)

RoPE is one such PE scheme that is primarily designed to capture relative positional information more effectively. It does this by applying a rotation to the query and key vectors based on their positions in the sections. The rotation is defined as follows:

$$\mathbf{Q}_R = Q \cdot R_{m,\theta}$$
$$\mathbf{K}_R = K \cdot R_{m,\theta}$$

Where $R_{m,\theta}$ is a rotation matrix that depends on the position m and a base angle θ. The rotation matrix for two dimensions is defined as:

$$R_{m,\theta} = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix}$$

$$\theta = 10000^{-2i/d_{\text{model}}}$$

The way that the rotation takes place for d_k dimensions is that you split it into pairs of two and then rotate each pair using the above rotation matrix. The analogy is how for 2D vectors you can first rotate one vector along one axis and then along another axis to get the final rotation. Similarly, for d_k dimensions, you can think of it as applying a series of rotations across different planes to rotate the vector.

3.1.1 How does it capture the Relative Positional Information?

From the above equations,

$$\mathbf{Q}_R \cdot \mathbf{K}_R^T = \left(Q \cdot R_{m,\theta}\right) \cdot \left(K \cdot R_{n,\theta}\right)^T$$

$$\mathbf{Q}_R \cdot \mathbf{K}_R^T = Q \cdot R_{m,\theta} \cdot R_{n,\theta}^T \cdot K^T$$

We know that $R_\alpha \cdot R_\beta = R_{\alpha+\beta}$ and $R_{-\alpha} = R_\alpha^T$. So,

$$\mathbf{Q}_R \cdot \mathbf{K}_R^T = Q \cdot \boxed{R_{m-n,\theta}} \cdot K^T$$

This shows that the attention weights depend only on the relative position $m - n$ between the query and key, rather than their absolute positions. This allows the model to capture relative positional information more effectively, as it can learn patterns based on the distance between tokens rather than their specific positions in the sequence.

Remark: Implementation using Complex Numbers.

Instead of doing heavy matrix multiplications with the rotation matrices, we can resort to complex numbers for rotations. After pairing up,

$$\mathbf{Q} = \begin{bmatrix} [Q_1, Q_2] \\ \vdots \\ [Q_{d_k-1}, Q_{d_k}] \end{bmatrix}$$

This can be viewed as a complex vector.

$$\mathbf{Q}_C = \begin{bmatrix} Q_1 + iQ_2 \\ \vdots \\ Q_{d_k-1} + iQ_{d_k} \end{bmatrix}$$

We know that multiplying a complex number by $e^{im\theta}$ results in a rotation of the complex number by an angle $m\theta$ in the complex plane. Therefore, we can represent the rotation as:

$$\mathbf{Q}_R = \mathbf{Q}_C \cdot e^{im\theta}$$

This approach is computationally efficient as it is just element-wise multiplication, and it avoids the need for explicit matrix multiplications with the rotation matrices. The same can be done for $\mathbf{K}$ as well.

3.2 Attention with Linear Biases. (ALiBi)

ALiBi is different from RoPE and AbsPE in the sense that it neither modifies the input embeddings nor does it apply any transformations to the query and key vectors. Instead, it adds a bias term directly to the attention scores based on the relative positions of the tokens.

$$\text{Standard Score} = \frac{\mathbf{QK}^T}{\sqrt{d_k}}$$

$$\text{ALiBi Score} = \frac{\mathbf{QK}^T}{\sqrt{d_k}} + m\,(i - j)$$

This goes to show that ALiBi adds more penalty to the attention scores as the distance between the query and key positions increases, which encourages the model to focus more on nearby tokens. Where,

$$m = 2^{\frac{-8h}{n}}$$

Implying that the bias factor or *slope* m reduces exponential for subsequent attention heads, allowing different heads to capture different ranges of dependencies. The first head will have the largest bias, focusing more on local context, while the later heads will have smaller biases, allowing them to attend to more distant tokens.

3.3 Relative Position Encoding. (RPE)